Statistical Measures

Sleep Disorder Analysis

Preethi Sri B -23MIS0470

Yuva Krithikka -23MIS0437

Madhumitha -23MSO360

Review 1:

1. Measuring Central Tendency(Mean,Median,Mode):

Code:

```
import pandas as pd

file_path = r"C:\Users\User\OneDrive\Documents\DataMining
Proj\subset_reduced_dataset.xlsx"
df = pd.read_excel(file_path)

print("----- Central Tendency Measures -----")

#Mean
print("\nMean values:")
print(df.mean(numeric_only=True))

#Median
print("\nMedian values:")
print(df.median(numeric_only=True))

# Mode
print("\nMode values:")
print(df.mode(numeric_only=True).iloc[0])
```

Output:

----- Central Tendency Measures -----

Mean values:

Age 0.486883

```
Gender              0.508000
Sleep_Disorder_Type    2.055000
AHI_Score           0.514212
SaO2_Level          0.500516
Diagnosis_Confirmed    0.884000
Smoking_Habit        0.474000
Alcohol_Consumption    1.012000
Exercise_Frequency    3.438000
Caffeine_Intake       1.010000
Work_Shift          0.971000
dtype: float64

Median values:
Age                 0.480519
Gender              1.000000
Sleep_Disorder_Type    3.000000
AHI_Score           0.527817
SaO2_Level          0.505613
Diagnosis_Confirmed    1.000000
Smoking_Habit        0.000000
Alcohol_Consumption    1.000000
Exercise_Frequency    3.000000
Caffeine_Intake       1.000000
Work_Shift          1.000000
dtype: float64

Mode values:
Age                 0.792208
Gender              1.000000
```

Sleep_Disorder_Type 3.000000

AHI_Score 0.129678

SaO2_Level 0.524860

Gender 1.000000

Sleep_Disorder_Type 3.000000

AHI_Score 0.129678

SaO2_Level 0.524860

Diagnosis_Confirmed 1.000000

Smoking_Habit 0.000000

Alcohol_Consumption 2.000000

Exercise_Frequency 4.000000

Caffeine_Intake 2.000000

Work_Shift 1.000000

Name: 0, dtype: float64

2. Range , Quartiles and Interquartile Range(IQR)

Code:

```
import pandas as pd

file_path = r"C:\Users\User\OneDrive\Documents\DataMining
Proj\subset_reduced_dataset.xlsx"
df = pd.read_excel(file_path)

print("----- Range, Quartiles & IQR -----")

# Range (max - min)
print("\nRange (Max - Min):")
range_values = df.max(numeric_only=True) - df.min(numeric_only=True)
print(range_values)

# Quartiles
print("\nQuartiles (Q1, Q2, Q3):")
Q1 = df.quantile(0.25, numeric_only=True)
Q2 = df.quantile(0.50, numeric_only=True)  # same as Median
Q3 = df.quantile(0.75, numeric_only=True)
print("Q1:\n", Q1)
```

```
print("Q2 (Median):\n", Q2)
print("Q3:\n", Q3)

# Interquartile Range (IQR = Q3 - Q1)
print("\nInterquartile Range (IQR):")
IQR = Q3 - Q1
print(IQR)
```

Output:

```
----- Range, Quartiles & IQR -----

Range (Max - Min):
Age                 1.0
Gender              1.0
Sleep_Disorder_Type    4.0
AHI_Score           1.0
SaO2_Level          1.0
Diagnosis_Confirmed    1.0
Smoking_Habit         1.0
Alcohol_Consumption    2.0
Exercise_Frequency     7.0
Caffeine_Intake       2.0
Work_Shift          2.0
dtype: float64

Quartiles (Q1, Q2, Q3):
Q1:
 Age                 0.233766
Gender              0.000000
Sleep_Disorder_Type    0.000000
AHI_Score           0.268761
SaO2_Level          0.254010
Diagnosis_Confirmed    1.000000
Smoking_Habit         0.000000
Alcohol_Consumption    0.000000
Exercise_Frequency     1.000000
Caffeine_Intake       0.000000
Work_Shift          0.000000
Name: 0.25, dtype: float64
Q2 (Median):
 Age                 0.480519
Gender              1.000000
Sleep_Disorder_Type    3.000000
AHI_Score           0.527817
```

```
SaO2_Level          0.505613
Diagnosis_Confirmed    1.000000
Smoking_Habit         0.000000
Alcohol_Consumption    1.000000
Exercise_Frequency     3.000000
Caffeine_Intake       1.000000
Work_Shift           1.000000
Name: 0.5, dtype: float64
Q3:
 Age                 0.740260
Gender               1.000000
Sleep_Disorder_Type    3.000000
AHI_Score            0.762157
SaO2_Level           0.753408
Diagnosis_Confirmed    1.000000
Smoking_Habit         1.000000
Alcohol_Consumption    2.000000
Exercise_Frequency     5.000000
Caffeine_Intake       2.000000
Work_Shift           2.000000
Name: 0.75, dtype: float64

Interquartile Range (IQR):
Age                 0.506494
Gender               1.000000
Sleep_Disorder_Type    3.000000
AHI_Score            0.493396
SaO2_Level           0.499399
Diagnosis_Confirmed    0.000000
Smoking_Habit         1.000000
Alcohol_Consumption    2.000000
Exercise_Frequency     4.000000
Caffeine_Intake       2.000000
Work_Shift           2.000000
dtype: float64
```

3. Five Number Summary

Code:

```
import pandas as pd

# Load the reduced dataset
file_path = r"C:\Users\User\OneDrive\Documents\DataMining
Proj\subset_reduced_dataset.xlsx"
df = pd.read_excel(file_path)

print("----- Five Number Summary -----")
```

```
five_num_summary = pd.DataFrame({
    "Minimum": df.min(numeric_only=True),
    "Q1 (25%)": df.quantile(0.25, numeric_only=True),
    "Median (Q2)": df.median(numeric_only=True),
    "Q3 (75%)": df.quantile(0.75, numeric_only=True),
    "Maximum": df.max(numeric_only=True)
})

print(five_num_summary)
```

Output:

```
----- Five Number Summary -----
                     Minimum  Q1 (25%)  Median (Q2)  Q3 (75%)  Maximum
Age                      0.0  0.233766     0.480519  0.740260      1.0
Gender                   0.0  0.000000     1.000000  1.000000      1.0
Sleep_Disorder_Type      0.0  0.000000     3.000000  3.000000      4.0
AHI_Score                0.0  0.268761     0.527817  0.762157      1.0
SaO2_Level               0.0  0.254010     0.505613  0.753408      1.0
Diagnosis_Confirmed      0.0  1.000000     1.000000  1.000000      1.0
Smoking_Habit            0.0  0.000000     0.000000  1.000000      1.0
Alcohol_Consumption      0.0  0.000000     1.000000  2.000000      2.0
Exercise_Frequency       0.0  1.000000     3.000000  5.000000      7.0
Caffeine_Intake          0.0  0.000000     1.000000  2.000000      2.0
Work_Shift               0.0  0.000000     1.000000  2.000000      2.0
```

4. Box Plots and Outliers:

Code:

```
import pandas as pd
import matplotlib.pyplot as plt

file_path = r"C:\Users\User\OneDrive\Documents\DataMining
Proj\subset_reduced_dataset.xlsx"
df = pd.read_excel(file_path)

numeric_cols = df.select_dtypes(include=['number']).columns

print("----- Outlier Detection using IQR -----")
outliers_dict = {}

for col in numeric_cols:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
```

```
    # Find outliers
    outliers = df[(df[col] < lower_bound) | (df[col] > upper_bound)][col]
    outliers_dict[col] = outliers.values

    print(f"\n{col}:")
    print(f"Lower Bound = {lower_bound}, Upper Bound = {upper_bound}")
    print(f"Outliers: {outliers.values if len(outliers) > 0 else 'None'}")

# Boxplots for all numeric columns
plt.figure(figsize=(12, 6))
df[numeric_cols].boxplot()
plt.title("Boxplots with Outliers (shown as dots)")
plt.xticks(rotation=45)
plt.show()
```

Output:

```
Matplotlib is building the font cache; this may take a moment.
----- Outlier Detection using IQR -----

Age:
Lower Bound = -0.525974025974026, Upper Bound = 1.5
Outliers: None

Gender:
Lower Bound = -1.5, Upper Bound = 2.5
Outliers: None

Sleep_Disorder_Type:
Lower Bound = -4.5, Upper Bound = 7.5
Outliers: None

AHI_Score:
Lower Bound = -0.4713327996798081, Upper Bound = 1.5022513508104867
Outliers: None

SaO2_Level:
Lower Bound = -0.4950882117080991, Upper Bound = 1.5025060144346436
Outliers: None

Diagnosis_Confirmed:
Lower Bound = 1.0, Upper Bound = 1.0
Outliers: [0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
```

0 0 0 0 0]

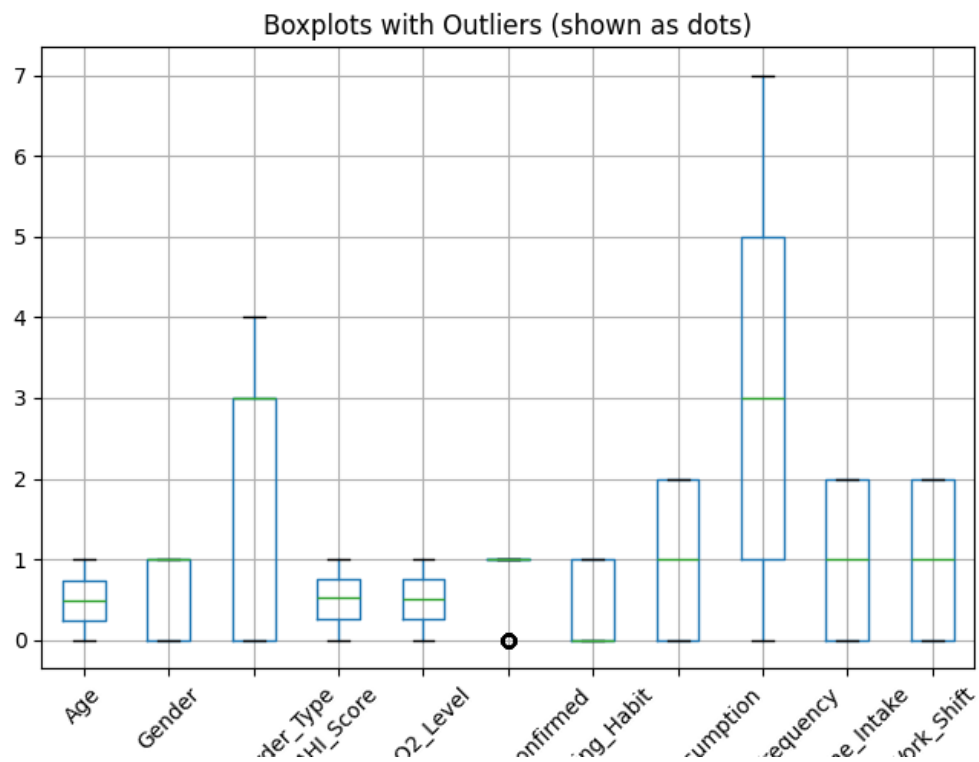
Smoking_Habit:
Lower Bound = -1.5, Upper Bound = 2.5
Outliers: None

Alcohol_Consumption:
Alcohol_Consumption:
Lower Bound = -3.0, Upper Bound = 5.0
Outliers: None

Exercise_Frequency:
Lower Bound = -5.0, Upper Bound = 11.0
Outliers: None

Caffeine_Intake:
Lower Bound = -3.0, Upper Bound = 5.0
Outliers: None

Work_Shift:
Lower Bound = -3.0, Upper Bound = 5.0
Outliers: None



5. Variance and Standard Deviation

Code:

```
import pandas as pd

file_path = r"C:\Users\User\OneDrive\Documents\DataMining
Proj\subset_reduced_dataset.xlsx"
df = pd.read_excel(file_path)


df_AF = df.iloc[:, 0:6]

print("----- Variance -----")
print(df_AF.var(numeric_only=True))

print("\n----- Standard Deviation -----")
print(df_AF.std(numeric_only=True))
```

Output:

```
Age                    0.086601
Gender                 0.250186
Sleep_Disorder_Type    2.290265
AHI_Score              0.082180
SaO2_Level             0.082866
Diagnosis_Confirmed    0.102647
dtype: float64

----- Standard Deviation -----
Age                    0.294281
Gender                 0.500186
Sleep_Disorder_Type    1.513362
AHI_Score              0.286670
SaO2_Level             0.287865
Diagnosis_Confirmed    0.320385
dtype: float64
```

6. Covariance and correlation

Code:

```
import pandas as pd

file_path = r"C:\Users\User\OneDrive\Documents\DataMining
Proj\subset_reduced_dataset.xlsx"
df = pd.read_excel(file_path)

df_AF = df.iloc[:, 0:6]

print("----- Covariance Matrix -----")
print(df_AF.cov())

print("\n----- Correlation Matrix -----")
print(df_AF.corr())
```

Output:

```
----- Covariance Matrix -----
                         Age    Gender  ...  SaO2_Level  Diagnosis_Confirmed
Age                 0.086601  0.002094  ...    0.001456            -0.000574
Gender              0.002094  0.250186  ...    0.005833            -0.011083
Sleep_Disorder_Type -0.006343 -0.007948  ...   -0.029732             0.008388
AHI_Score           0.000773 -0.000813  ...   -0.002168             0.043055
SaO2_Level          0.001456  0.005833  ...    0.082866            -0.032164
Diagnosis_Confirmed -0.000574 -0.011083  ...   -0.032164             0.102647

[6 rows x 6 columns]

----- Correlation Matrix -----
                         Age    Gender  ...  SaO2_Level  Diagnosis_Confirmed
Age                 1.000000  0.014226  ...    0.017190            -0.006089
Gender              0.014226  1.000000  ...    0.040510            -0.069160
Sleep_Disorder_Type -0.014243 -0.010500  ...   -0.068248             0.017301
AHI_Score           0.009158 -0.005671  ...   -0.026270             0.468780
SaO2_Level          0.017190  0.040510  ...    1.000000            -0.348742
Diagnosis_Confirmed -0.006089 -0.069160  ...   -0.348742             1.000000
```

7. Chi-Square test

Code:

```
import pandas as pd
from scipy.stats import chi2_contingency

file_path = r"C:\Users\User\OneDrive\Documents\DataMining
Proj\subset_reduced_dataset.xlsx"
df = pd.read_excel(file_path)

# Assuming A and B are categorical
contingency_table = pd.crosstab(df.iloc[:,0], df.iloc[:,1])
chi2, p, dof, expected = chi2_contingency(contingency_table)

print("Chi-Square Test Results:")
print("Chi2 Value:", chi2)
print("p-value:", p)
print("Degrees of Freedom:", dof)
print("Expected Frequencies:\n", expected)
```

Output:

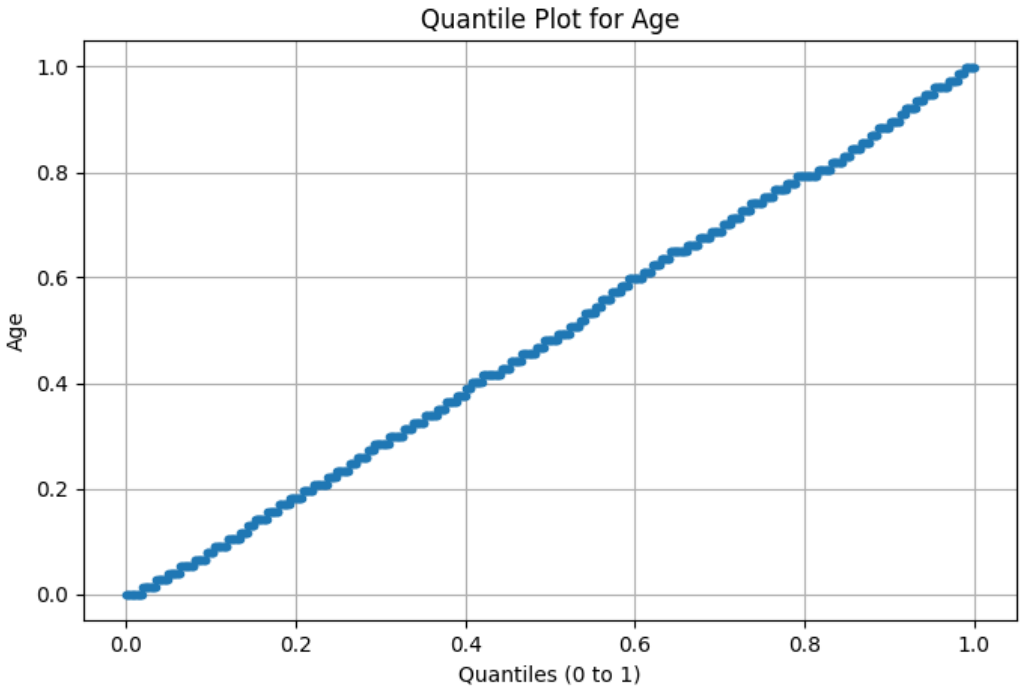
```
Chi-Square Test Results:
Chi2 Value: 73.7894421072609
p-value: 0.5825637929794798
Degrees of Freedom: 77
```

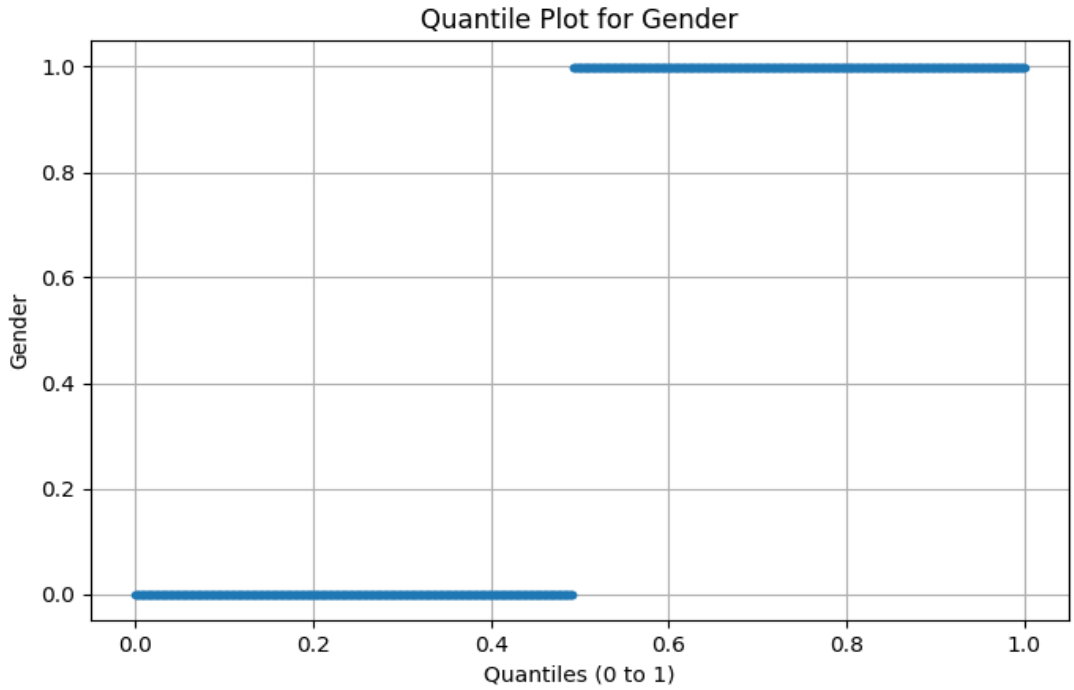
8. Quantile plot

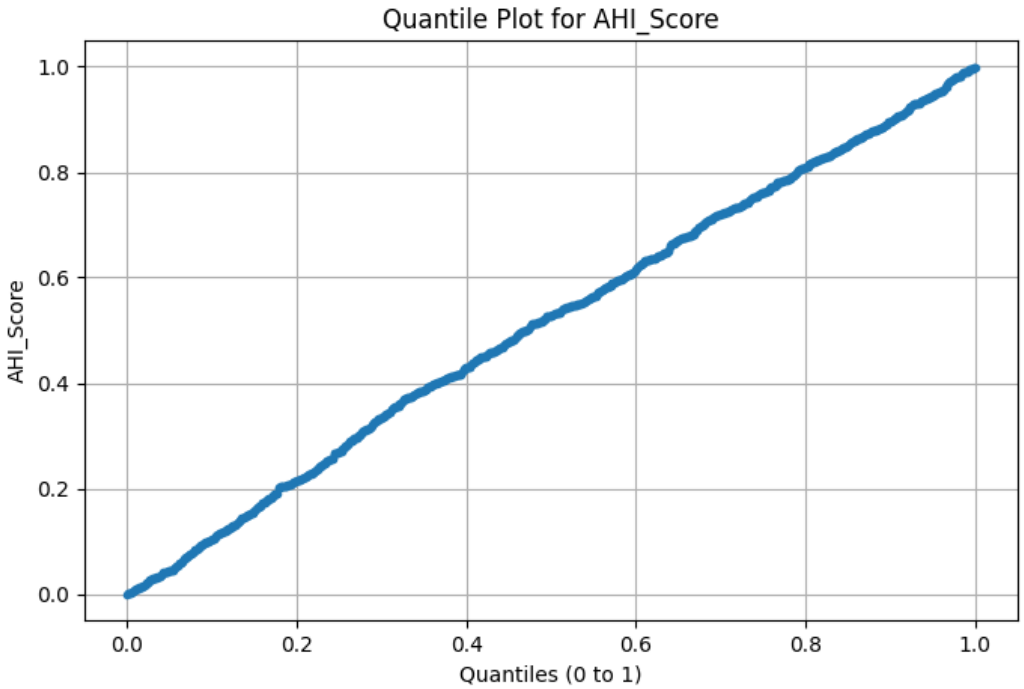
Code:

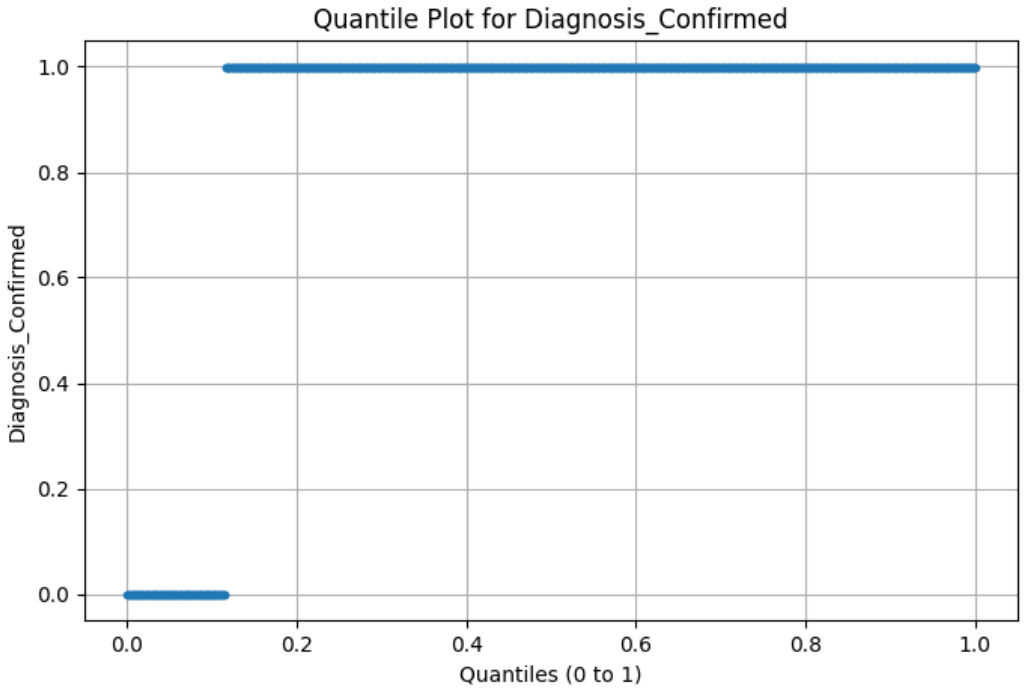
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Load dataset
file_path = r"C:\Users\User\OneDrive\Documents\DataMining
Proj\subset_reduced_dataset.xlsx"
df = pd.read_excel(file_path)

# Select only columns A-F
df_selected = df.iloc[:, 0:6]

# Generate quantile plots
for column in df_selected.columns:
    data = df_selected[column].dropna()
    quantiles = np.linspace(0, 1, len(data))
    sorted_data = np.sort(data)

    plt.figure(figsize=(8, 5))
    plt.plot(quantiles, sorted_data, 'o', markersize=3)
    plt.title(f"Quantile Plot for {column}")
    plt.xlabel("Quantiles (0 to 1)")
    plt.ylabel(column)
    plt.grid(True)
    plt.show()
```

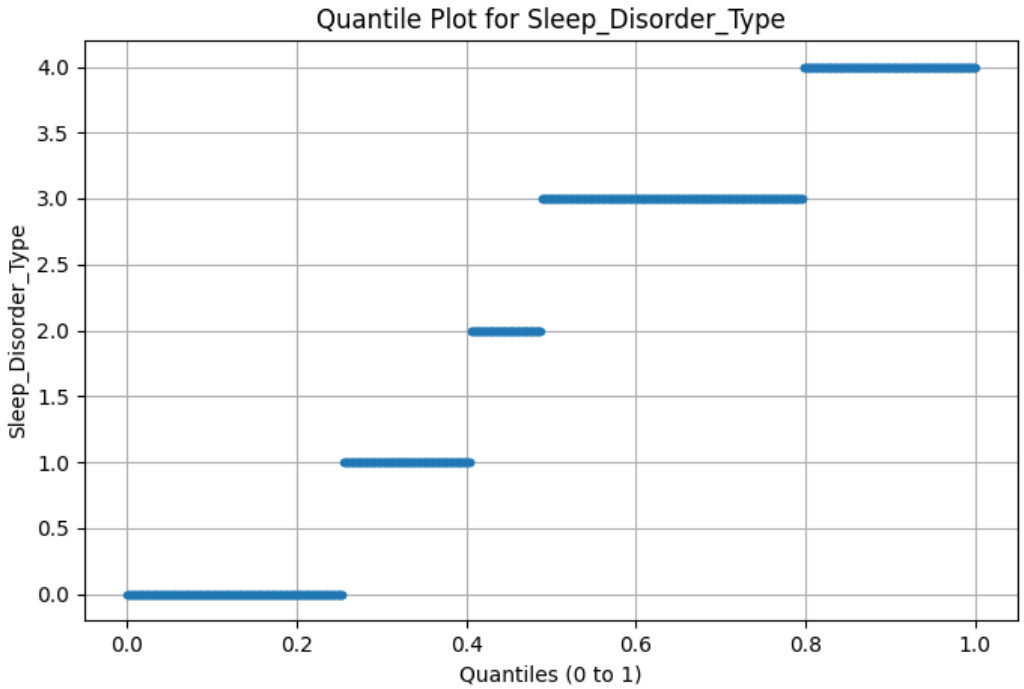
Output:

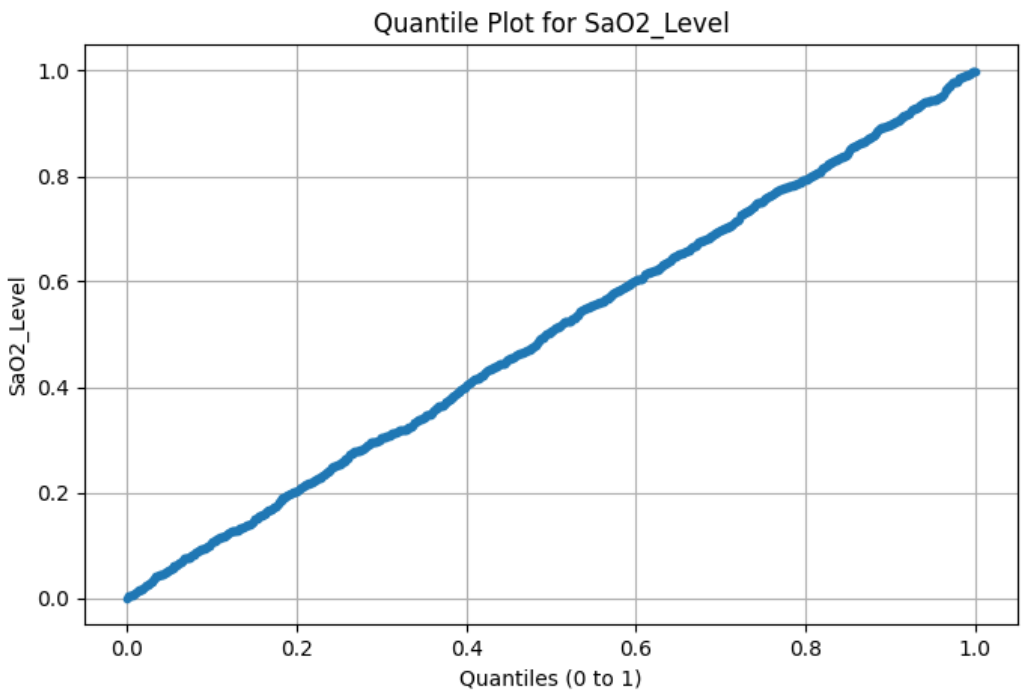

Quantile Plot for Age
Age
Quantiles (0 to 1)
0.0
0.2
0.4
0.6
0.8
1.0


Quantile Plot for Gender
Gender
Quantiles (0 to 1)
0.0
0.2
0.4
0.6
0.8
1.0


Quantile Plot for AHI_Score
AHI_Score
Quantiles (0 to 1)
0.0
0.2
0.4
0.6
0.8
1.0


Quantile Plot for Diagnosis_Confirmed
Diagnosis_Confirmed
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0
Quantiles (0 to 1)


Quantile Plot for Sleep_Disorder_Type
Sleep_Disorder_Type
4.0
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
0.0
0.2
0.4
0.6
0.8
1.0
Quantiles (0 to 1)

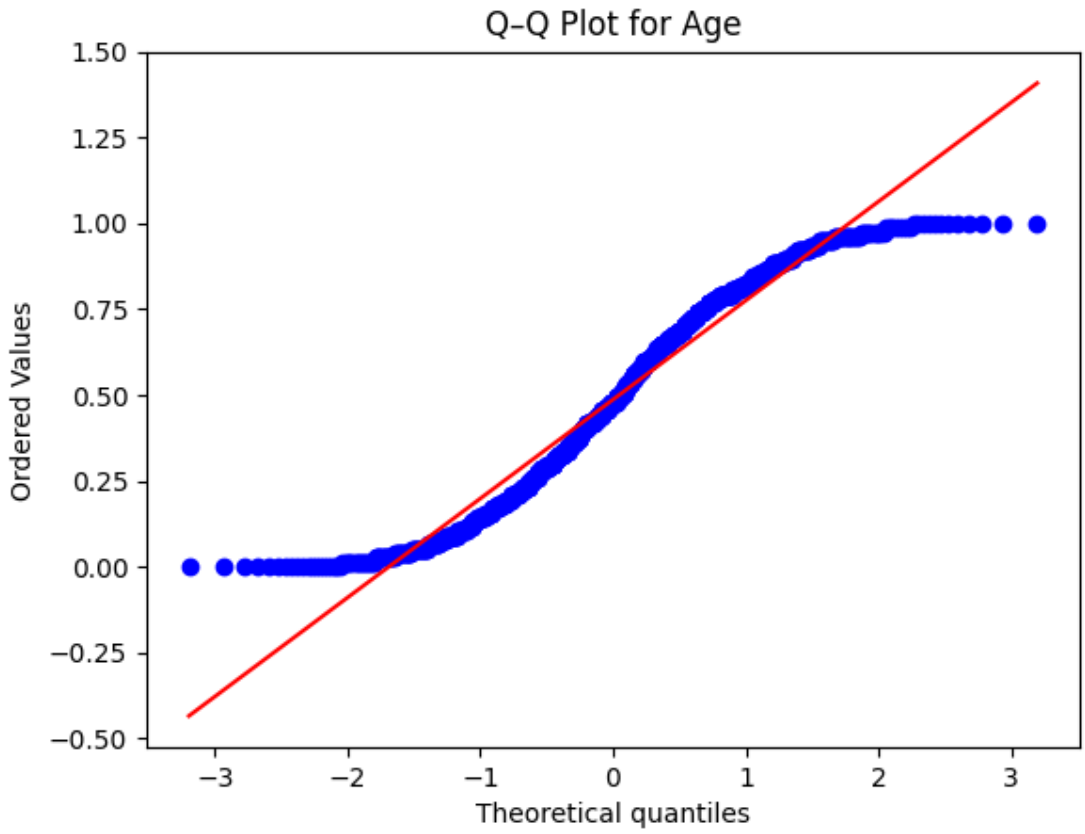


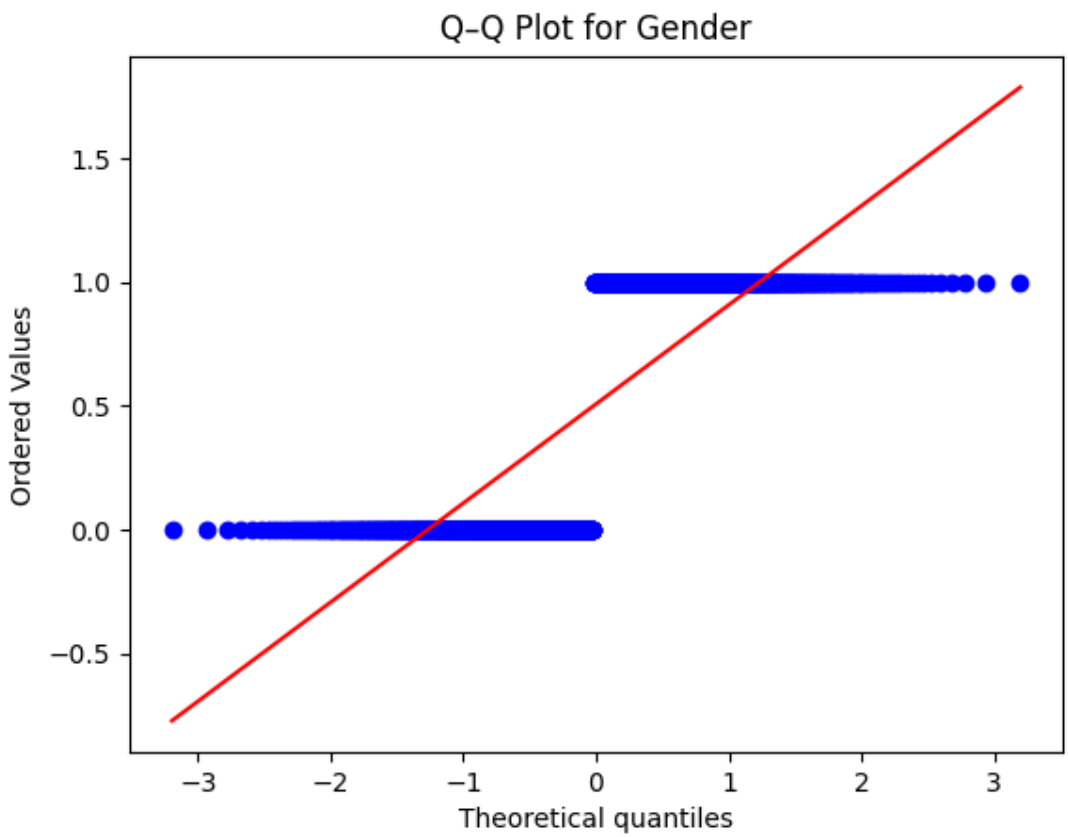
9. Q-Q Plot

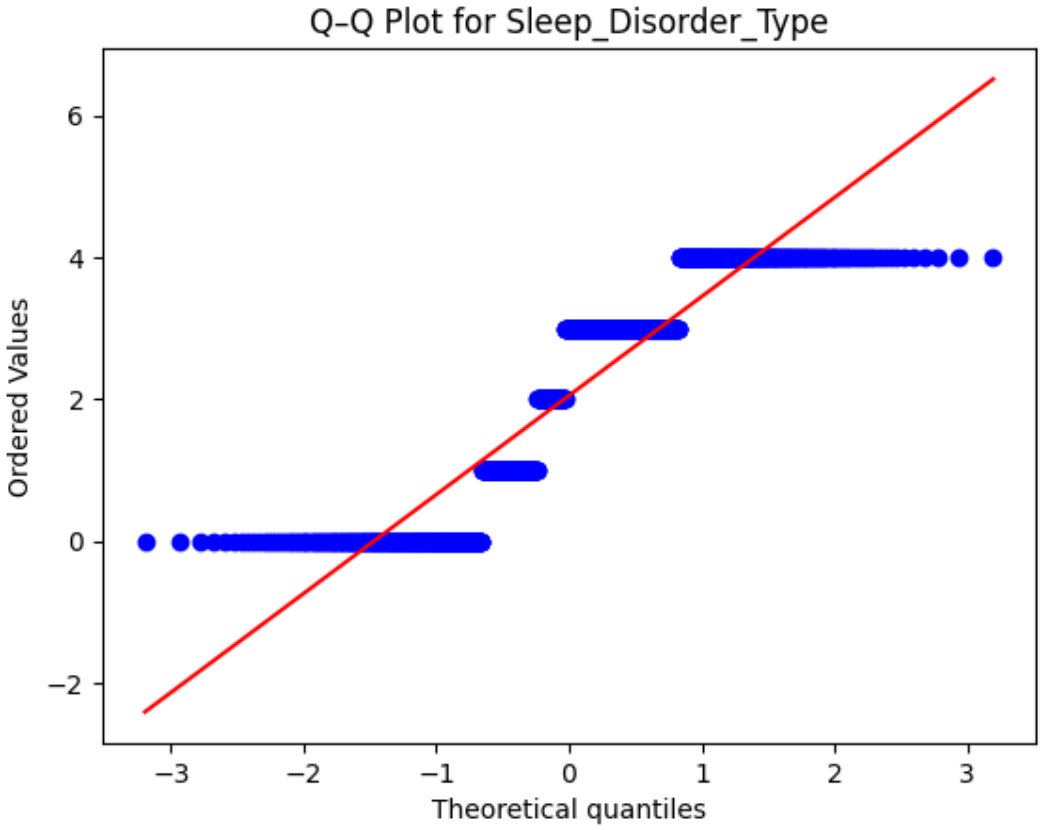
Code:

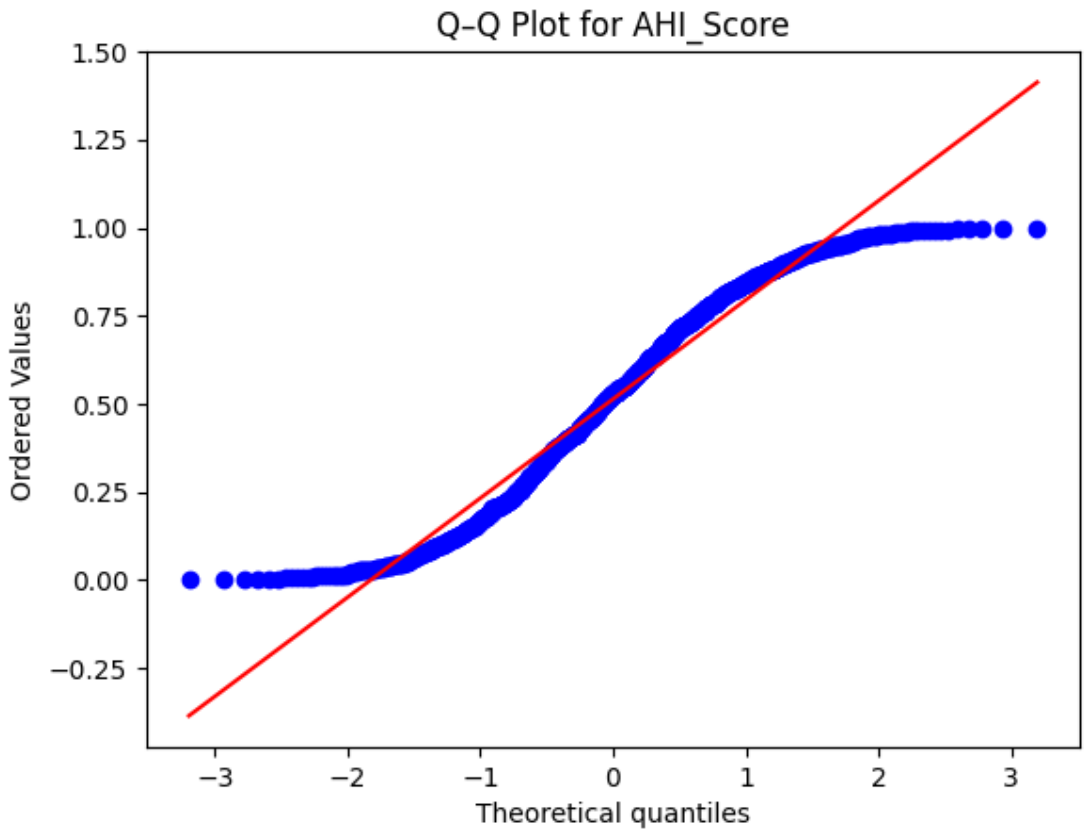
```
import pandas as pd
import matplotlib.pyplot as plt
import scipy.stats as stats

file_path = r"C:\Users\User\OneDrive\Documents\DataMining
Proj\subset_reduced_dataset.xlsx"
df = pd.read_excel(file_path)

df_AF = df.iloc[:, 0:6]

for col in df_AF.columns:
    stats.probplot(df_AF[col], dist="norm", plot=plt)
    plt.title(f"Q-Q Plot for {col}")
    plt.show()
```

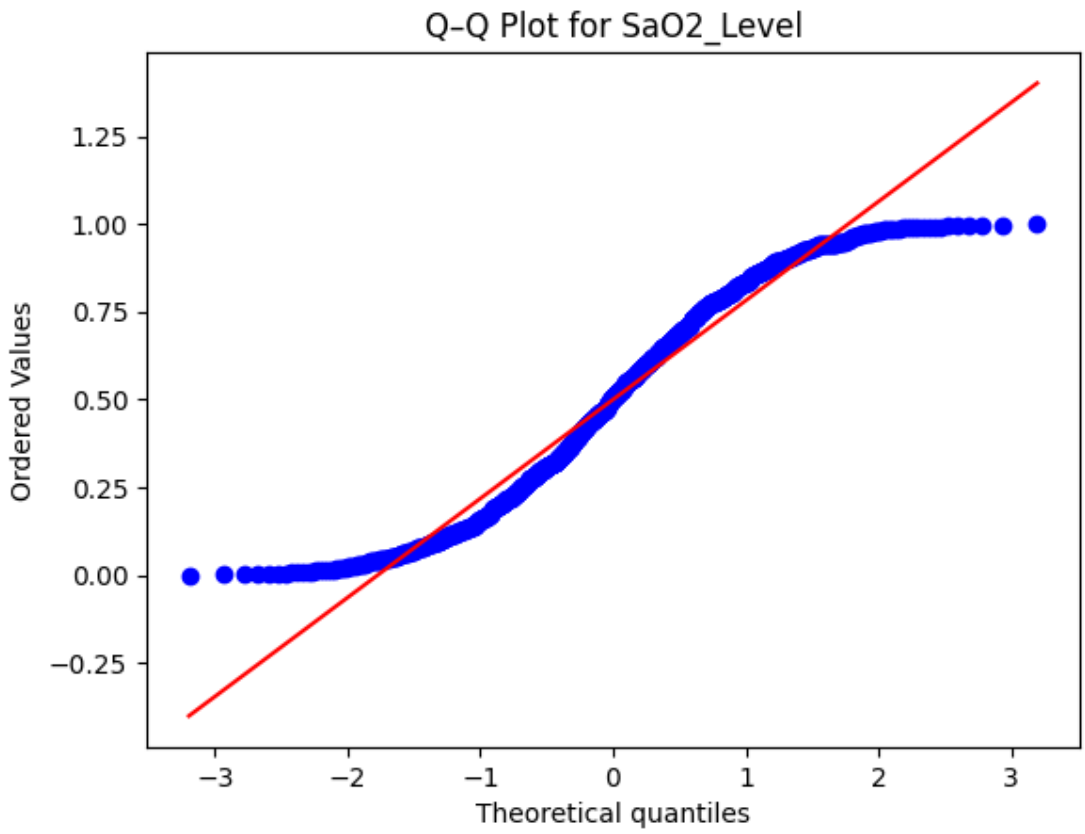
Output:

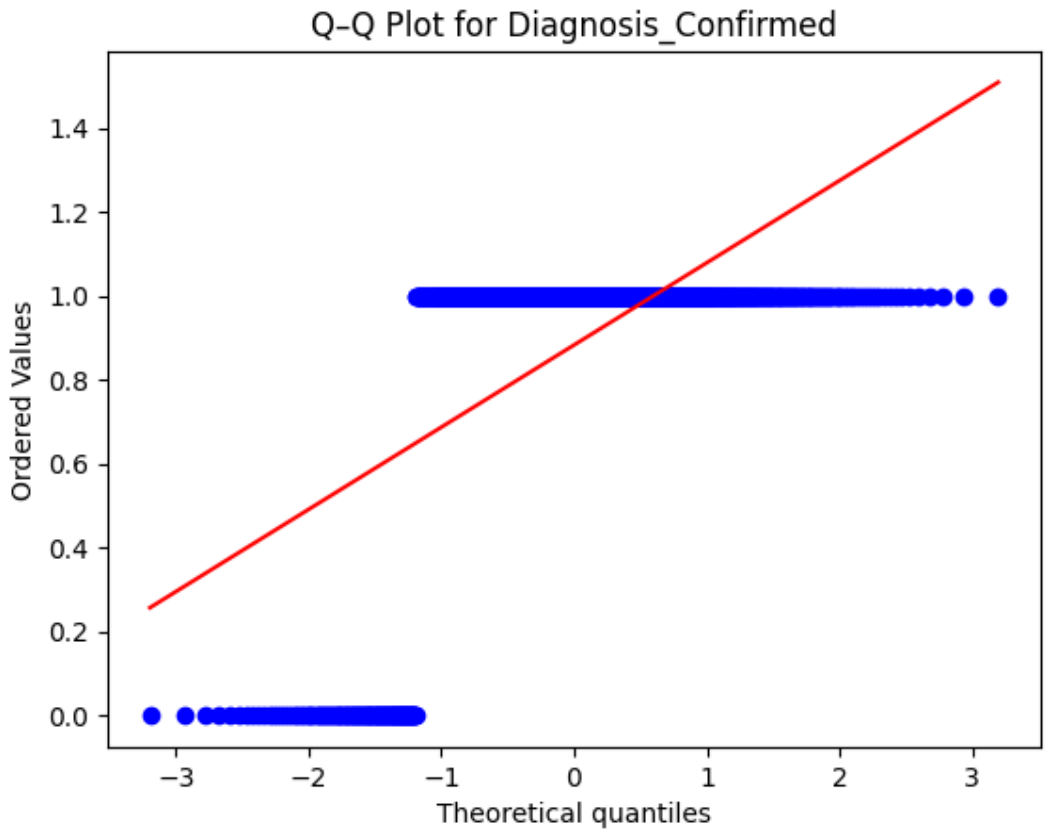

Q–Q Plot for Age
Ordered Values
Theoretical quantiles
1.50
1.25
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−3
−2
−1
0
1
2
3


Q–Q Plot for Gender
Ordered Values
Theoretical quantiles
1.5
1.0
0.5
0.0
−0.5
−3
−2
−1
0
1
2
3


Q–Q Plot for Sleep_Disorder_Type
Ordered Values
Theoretical quantiles
6
4
2
0
−2
−3
−2
−1
0
1
2
3


Q–Q Plot for AHI_Score
Ordered Values
Theoretical quantiles
1.50
1.25
1.00
0.75
0.50
0.25
0.00
−0.25
−3
−2
−1
0
1
2
3


Q–Q Plot for SaO2_Level
Ordered Values
Theoretical quantiles
1.25
1.00
0.75
0.50
0.25
0.00
−0.25
−3
−2
−1
0
1
2
3

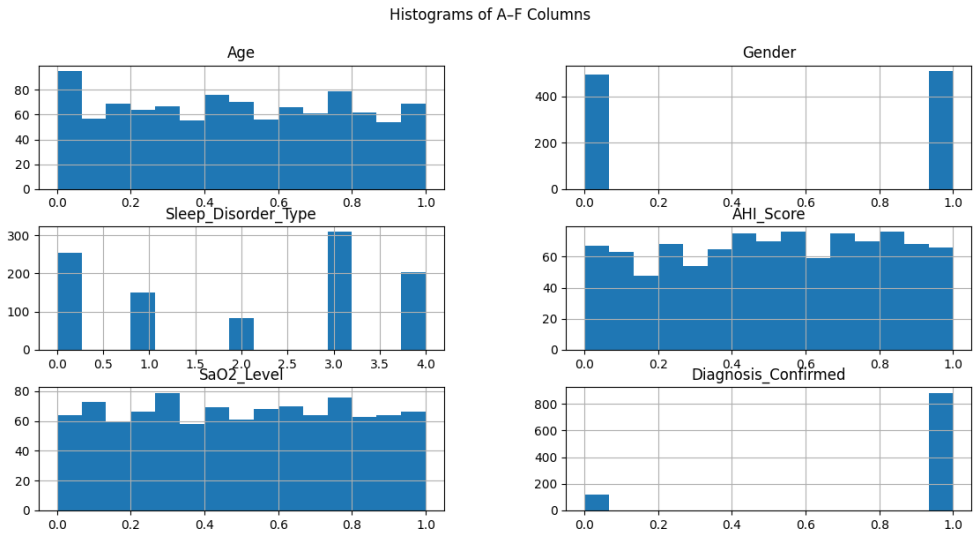

Q–Q Plot for Diagnosis_Confirmed
Ordered Values
Theoretical quantiles
1.4
1.2
1.0
0.8
0.6
0.4
0.2
0.0
−3
−2
−1
0
1
2
3

10. Histogram

Code:

```
import pandas as pd
import matplotlib.pyplot as plt

file_path = r"C:\Users\User\OneDrive\Documents\DataMining
Proj\subset_reduced_dataset.xlsx"
df = pd.read_excel(file_path)

df_AF = df.iloc[:, 0:6]

df_AF.hist(figsize=(12, 8), bins=15)
plt.suptitle("Histograms of A-F Columns")
plt.show()
```

Output:



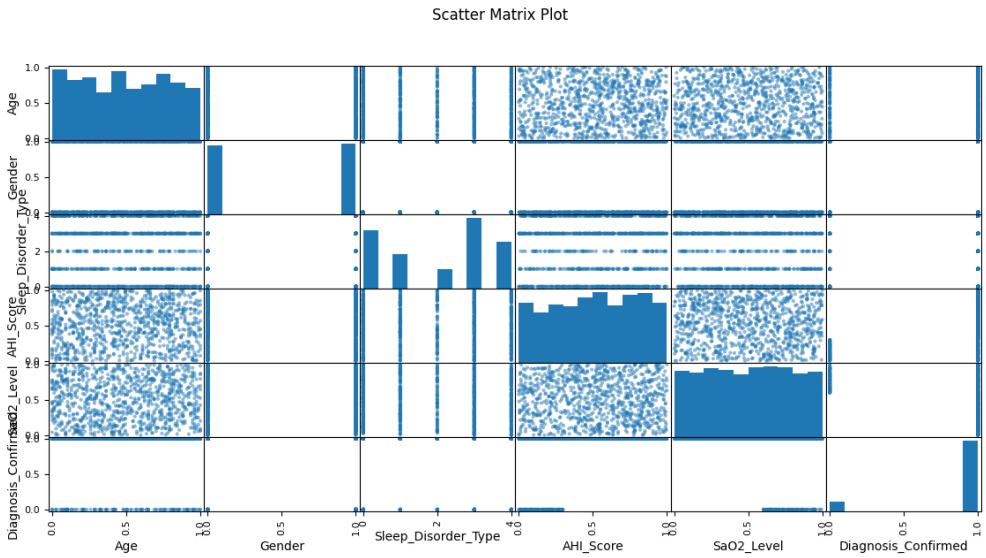
11. Scatter Plot

Code:

```
import pandas as pd
import matplotlib.pyplot as plt

file_path = r"C:\Users\User\OneDrive\Documents\DataMining
Proj\subset_reduced_dataset.xlsx"
df = pd.read_excel(file_path)

df_AF = df.iloc[:, 0:6]

pd.plotting.scatter_matrix(df_AF, figsize=(12, 8), diagonal='hist')
plt.suptitle("Scatter Matrix Plot")
```

```
plt.show()
```

Output:



12. PEARSON CORRELATION COEFFICIENT and COSINE SIMILARITY

Code:

```
import pandas as pd
from sklearn.metrics.pairwise import cosine_similarity

file_path = r"C:\Users\User\OneDrive\Documents\DataMining
Proj\subset_reduced_dataset.xlsx"
df = pd.read_excel(file_path)

df_AF = df.iloc[:, 0:6]

print("----- Pearson Correlation -----")
print(df_AF.corr(method='pearson'))

print("\n----- Cosine Similarity -----")
cos_sim = cosine_similarity(df_AF.T)  # Transposed for column-wise similarity
cosine_df = pd.DataFrame(cos_sim, index=df_AF.columns, columns=df_AF.columns)
print(cosine_df)
```

Output:

```
----- Pearson Correlation -----
                         Age    Gender  ...  SaO2_Level  Diagnosis_Confirmed
Age                 1.000000  0.014226  ...    0.017190            -0.006089
Gender              0.014226  1.000000  ...    0.040510            -0.069160
Sleep_Disorder_Type -0.014243 -0.010500  ...   -0.068248             0.017301
AHI_Score           0.009158 -0.005671  ...   -0.026270             0.468780
SaO2_Level          0.017190  0.040510  ...    1.000000            -0.348742
Diagnosis_Confirmed -0.006089 -0.069160  ...   -0.348742             1.000000

[6 rows x 6 columns]

----- Cosine Similarity -----
                         Age    Gender  ...  SaO2_Level  Diagnosis_Confirmed
Age                 1.000000  0.615220  ...    0.746494             0.803688
Gender              0.615220  1.000000  ...    0.632081             0.653606
Sleep_Disorder_Type 0.684966  0.569645  ...    0.678051             0.760699
AHI_Score           0.749999  0.620672  ...    0.750955             0.899029
SaO2_Level          0.746494  0.632081  ...    1.000000             0.755934
Diagnosis_Confirmed 0.803688  0.653606  ...    0.755934             1.000000

[6 rows x 6 columns]
```

13. MINKOWSHKI DISTANCE

Code:

```
import pandas as pd
from scipy.spatial import distance

file_path = r"C:\Users\User\OneDrive\Documents\DataMining
Proj\subset_reduced_dataset.xlsx"
df = pd.read_excel(file_path)

df_AF = df.iloc[:, 0:6]

row1 = df_AF.iloc[0]
row2 = df_AF.iloc[1]

minkowski_dist = distance.minkowski(row1, row2, p=3)
print("Minkowski Distance (p=3) between Row1 & Row2:", minkowski_dist)
```

Output:

```
Minkowski Distance (p=3) between Row1 & Row2: 0.4940014661133043
```

14. DISTANCE MEASURES FOR DISSIMILARITY (EUCLIDEAN / MANHATTAN / SUPREMUM)

Code:

```
import pandas as pd
from scipy.spatial import distance
```

```
file_path = r"C:\Users\User\OneDrive\Documents\DataMining
Proj\subset_reduced_dataset.xlsx"
df = pd.read_excel(file_path)

df_AF = df.iloc[:, 0:6]

row1 = df_AF.iloc[0]
row2 = df_AF.iloc[1]

euclidean = distance.euclidean(row1, row2)
manhattan = distance.cityblock(row1, row2)
supremum = distance.chebyshev(row1, row2)

print("Euclidean Distance:", euclidean)
print("Manhattan Distance:", manhattan)
print("Supremum (Chebyshev) Distance:", supremum)
```

Output:

```
Euclidean Distance: 0.5278025128305488
Manhattan Distance: 0.746778113004505
Supremum (Chebyshev) Distance: 0.48051948051948057
```

15. DISSIMILARITY MATRIX

Code:

```
import pandas as pd
from scipy.spatial.distance import pdist, squareform

file_path = r"C:\Users\User\OneDrive\Documents\DataMining
Proj\subset_reduced_dataset.xlsx"
df = pd.read_excel(file_path)

df_AF = df.iloc[:, 0:6]

# Euclidean Dissimilarity Matrix
euclidean_matrix = squareform(pdist(df_AF, metric='euclidean'))
print("Euclidean Dissimilarity Matrix:\n", euclidean_matrix)

# Manhattan Dissimilarity Matrix
manhattan_matrix = squareform(pdist(df_AF, metric='cityblock'))
print("\nManhattan Dissimilarity Matrix:\n", manhattan_matrix)

# Supremum (Chebyshev) Dissimilarity Matrix
supremum_matrix = squareform(pdist(df_AF, metric='chebyshev'))
print("\nSupremum (Chebyshev) Dissimilarity Matrix:\n", supremum_matrix)
```

Output:

```
Euclidean Dissimilarity Matrix:
 [[0.         0.52780251 4.20553317 ... 1.60766132 4.06846346 1.03259676]
 [0.52780251 0.         4.24304233 ... 1.72192103 4.10054988 0.71813008]
 [4.20553317 4.24304233 0.         ... 3.00231509 1.11785133 4.2081435 ]
 ...
 [1.60766132 1.72192103 3.00231509 ... 0.         3.1896826  1.66914412]
 [4.06846346 4.10054988 1.11785133 ... 3.1896826  0.         4.08969082]
 [1.03259676 0.71813008 4.2081435  ... 1.66914412 4.08969082 0.        ]]

Manhattan Dissimilarity Matrix:
 [[0.         0.74677811 6.36286388 ... 3.28283478 4.96301645 1.71259444]
 [0.74677811 0.         6.6869884  ... 3.6069593  5.28714097 1.07567999]
 [6.36286388 6.6869884  0.         ... 3.15010898 1.84170789 6.09294804]
 ...
 [3.28283478 3.6069593  3.15010898 ... 0.         4.71757294 3.24305702]
 [4.96301645 5.28714097 1.84170789 ... 4.71757294 0.         5.44114944]
 [1.71259444 1.07567999 6.09294804 ... 3.24305702 5.44114944 0.        ]]

Supremum (Chebyshev) Dissimilarity Matrix:
 [[0.         0.48051948 4.         ... 1.         4.         0.78727236]
 [0.48051948 0.         4.         ... 1.         4.         0.57594557]
 [4.         4.         0.         ... 3.         1.         4.        ]
 ...
 [1.         1.         3.         ... 0.         3.         1.        ]
 [4.         4.         1.         ... 3.         0.         4.        ]
 [0.78727236 0.57594557 4.         ... 1.         4.         0.        ]]
```

16. SIMMILARITY AND DISSIMILARITY FOR(ORDINAL,NOMINAL,BINARY) .

Code:

```
import pandas as pd
import numpy as np
from scipy.spatial.distance import pdist, squareform
from sklearn.preprocessing import LabelEncoder

file_path = r"C:\Users\User\OneDrive\Documents\DataMining
Proj\subset_reduced_dataset.xlsx"
df = pd.read_excel(file_path)

df_selected = df.iloc[:, 0:6]

df_nominal = df_selected.astype(str).apply(LabelEncoder().fit_transform)
def simple_matching(u, v):
    return np.sum(u != v) / len(u)
nominal_dist = squareform(pdist(df_nominal.values, metric=simple_matching))
print("\nNominal/Categorical (Simple Matching):")
print(pd.DataFrame(nominal_dist).round(2))
```

```
df_binary = df_selected.applymap(lambda v: 1 if v >
df_selected.median().median() else 0)
binary_dist = squareform(pdist(df_binary.values, metric="jaccard"))
print("\nBinary (Jaccard Distance):")
print(pd.DataFrame(binary_dist).round(2))

df_ordinal = df_selected.rank(method="average") /
df_selected.rank(method="average").max()
ordinal_dist = squareform(pdist(df_ordinal.values, metric="euclidean"))
print("\nOrdinal (Euclidean on Ranks):")
print(pd.DataFrame(ordinal_dist).round(2))
```

Output:

```
Nominal/Categorical (Simple Matching):
      0     1     2     3     4     5     6    ...   993   994   995   996   997   998   999
0    0.00  0.50  0.83  0.67  0.67  0.83  0.67  ...  0.67  0.67  0.67  0.67  0.83  0.67  0.50
1    0.50  0.00  0.83  0.67  0.67  0.83  0.67  ...  0.67  0.50  0.67  0.67  0.83  0.67  0.50
2    0.83  0.83  0.00  0.67  0.67  0.50  0.67  ...  0.67  0.83  0.67  0.83  0.67  0.67  0.83
3    0.67  0.67  0.67  0.00  0.83  0.67  0.50  ...  0.50  0.83  0.83  0.83  0.67  0.83  0.67
4    0.67  0.67  0.67  0.83  0.00  0.67  0.83  ...  0.83  0.67  0.33  0.67  0.83  0.50  0.67
..    ...   ...   ...   ...   ...   ...   ...  ...   ...   ...   ...   ...   ...   ...   ...
995  0.67  0.67  0.67  0.83  0.33  0.67  0.83  ...  0.83  0.67  0.00  0.67  0.83  0.50  0.67
996  0.67  0.67  0.83  0.83  0.67  0.83  0.83  ...  0.83  0.67  0.67  0.00  0.83  0.67  0.67
997  0.83  0.83  0.67  0.67  0.83  0.67  0.67  ...  0.67  0.67  0.83  0.83  0.00  0.83  0.83
998  0.67  0.67  0.67  0.83  0.50  0.67  0.83  ...  0.83  0.67  0.50  0.67  0.83  0.00  0.67
999  0.50  0.50  0.83  0.67  0.67  0.83  0.67  ...  0.67  0.67  0.67  0.67  0.83  0.67  0.00

[1000 rows x 1000 columns]
```

```
Binary (Jaccard Distance):
      0     1     2     3     4     5     6    ...   993   994   995   996   997   998   999
0    0.00  0.33  0.80  0.60  0.67  0.80  0.40  ...  0.25  0.50  0.33  0.00  0.60  0.75  0.33
1    0.33  0.00  0.75  0.50  0.50  0.75  0.60  ...  0.50  0.33  0.67  0.33  0.50  0.67  0.00
2    0.80  0.75  0.00  0.25  0.67  0.00  0.40  ...  0.60  0.80  0.75  0.80  0.25  0.75  0.75
3    0.60  0.50  0.25  0.00  0.75  0.25  0.20  ...  0.40  0.60  0.80  0.60  0.00  0.80  0.50
4    0.67  0.50  0.67  0.75  0.00  0.67  0.80  ...  0.75  0.67  0.50  0.67  0.75  0.50  0.50
..    ...   ...   ...   ...   ...   ...   ...  ...   ...   ...   ...   ...   ...   ...   ...
995  0.33  0.67  0.75  0.80  0.50  0.75  0.60  ...  0.50  0.75  0.00  0.33  0.80  0.67  0.67
996  0.00  0.33  0.80  0.60  0.67  0.80  0.40  ...  0.25  0.50  0.33  0.00  0.60  0.75  0.33
997  0.60  0.50  0.25  0.00  0.75  0.25  0.20  ...  0.40  0.60  0.80  0.60  0.00  0.80  0.50
998  0.75  0.67  0.75  0.80  0.50  0.75  0.83  ...  0.80  0.33  0.67  0.75  0.80  0.00  0.67
999  0.33  0.00  0.75  0.50  0.50  0.75  0.60  ...  0.50  0.33  0.67  0.33  0.50  0.67  0.00

[1000 rows x 1000 columns]
```

```
Ordinal (Euclidean on Ranks):
      0     1     2     3     4     5     6    ...   993   994   995   996   997   998   999
0    0.00  0.53  1.38  0.79  1.21  1.22  0.78  ...  0.72  1.18  0.97  0.59  1.08  1.15  1.03
1    0.53  0.00  1.47  0.97  1.06  1.41  1.11  ...  0.85  1.03  0.93  0.76  1.22  1.25  0.71
2    1.38  1.47  0.00  0.97  0.93  0.46  1.14  ...  1.19  1.22  1.13  1.04  0.58  0.83  1.36
3    0.79  0.97  0.97  0.00  1.27  0.88  0.36  ...  0.39  1.14  1.22  0.88  0.49  1.16  1.10
4    1.21  1.06  0.93  1.27  0.00  1.05  1.47  ...  1.31  0.73  0.60  0.76  1.10  0.59  0.89
..    ...   ...   ...   ...   ...   ...   ...  ...   ...   ...   ...   ...   ...   ...   ...
995  0.97  0.93  1.13  1.22  0.60  0.98  1.29  ...  1.12  1.00  0.00  0.48  1.24  0.66  1.15
996  0.59  0.76  1.04  0.88  0.76  0.86  0.92  ...  0.85  0.90  0.48  0.00  0.96  0.60  1.02
997  1.08  1.22  0.58  0.49  1.10  0.68  0.73  ...  0.82  1.07  1.24  0.96  0.00  0.98  1.12
998  1.15  1.25  0.83  1.16  0.59  0.79  1.26  ...  1.24  0.82  0.66  0.60  0.98  0.00  1.20
999  1.03  0.71  1.36  1.10  0.89  1.50  1.39  ...  1.15  0.64  1.15  1.02  1.12  1.20  0.00

[1000 rows x 1000 columns]
```

17. MIXED ATTRIBUTES

Code:

```python
import pandas as pd
import gower
import numpy as np

file_path = "subset_reduced_dataset.xlsx"
df = pd.read_excel(file_path)
data = df.iloc[:, 0:6]

# Compute Gower distance (dissimilarity) matrix
gower_dist = gower.gower_matrix(data)

# Round for better readability
gower_dist = np.round(gower_dist, 2)
print(pd.DataFrame(gower_dist))
```

Output:

```
      0     1     2     3     4     5     6    ...   993   994   995   996   997   998   999
0    0.00  0.12  0.56  0.27  0.41  0.48  0.27  ...  0.24  0.35  0.28  0.15  0.42  0.33  0.29
1    0.12  0.00  0.61  0.33  0.33  0.55  0.39  ...  0.31  0.25  0.25  0.24  0.48  0.38  0.18
2    0.56  0.61  0.00  0.29  0.29  0.09  0.33  ...  0.39  0.50  0.42  0.43  0.15  0.31  0.52
3    0.27  0.33  0.29  0.00  0.50  0.22  0.09  ...  0.11  0.45  0.46  0.31  0.15  0.42  0.38
3    0.27  0.33  0.29  0.00  0.50  0.22  0.09  ...  0.11  0.45  0.46  0.31  0.15  0.42  0.38
4    0.41  0.33  0.29  0.50  0.00  0.36  0.59  ...  0.52  0.23  0.13  0.26  0.43  0.17  0.24
..    ...   ...   ...   ...   ...   ...   ...  ...   ...   ...   ...   ...   ...   ...   ...
995  0.28  0.25  0.42  0.46  0.13  0.34  0.47  ...  0.39  0.32  0.00  0.15  0.53  0.18  0.34
996  0.15  0.24  0.43  0.31  0.26  0.35  0.34  ...  0.29  0.29  0.15  0.00  0.38  0.20  0.30
997  0.42  0.48  0.15  0.15  0.43  0.20  0.19  ...  0.25  0.36  0.53  0.38  0.00  0.41  0.42
998  0.33  0.38  0.31  0.42  0.17  0.28  0.51  ...  0.49  0.28  0.18  0.20  0.41  0.00  0.41
999  0.29  0.18  0.52  0.38  0.24  0.59  0.47  ...  0.40  0.19  0.34  0.30  0.42  0.41  0.00

[1000 rows x 1000 columns]
```